

EXECUTIVE SUMMARY

Project: virtual_project.zip

Technical Debt Score: 100/100 (Unknown)

Project Fingerprint: N/A [ID]

Generated by Zip2Pdf Pro AI at 2026-06-04 08:00

AI QUICK ADVICE

Project looks clean. Good job!

Logo.jpg

[Binary File - Skipped]

ZipToPDF_Pro_v2.5.spec

```
1  #-*- mode: python ; coding: utf-8 -*-
2
3
4  block_cipher = None
5
6
7  a = Analysis(
8      ['zip_to_pdf.py'],
9      pathex=[],
10     binaries=[],
11     datas=[('Logo.jpg', '.'), ('app.ico', '.')],
12     hiddenimports=[],
13     hookspath=[],
14     hooksconfig={},
15     runtime_hooks=[],
16     excludes=[],
17     win_no_prefer_redirects=False,
18     win_private_assemblies=False,
19     cipher=block_cipher,
20     noarchive=False,
21 )
22 pyz = PYZ(a.pure, a.zipped_data, cipher=block_cipher)
23
24 exe = EXE(
25     pyz,
26     a.scripts,
27     a.binaries,
```

```
28  a.zipfiles,
29  a.datas,
30  [],
31  name='ZipToPDF_Pro_v2.5',
32  debug=False,
33  bootloader_ignore_signals=False,
34  strip=False,
35  upx=True,
36  upx_exclude=[],
37  runtime_tmpdir=None,
38  console=False,
39  disable_windowed_traceback=False,
40  argv_emulation=False,
41  target_arch=None,
42  codesign_identity=None,
43  entitlements_file=None,
44  icon=['app.ico'],
45 )
```

app_config.json

```
1 {
2   "show_welcome": false,
3   "manual_update_only": false,
4   "offline_mode": false,
5   "api_key": "AlzaSyCM9PxLOZ3DLTZdmUN6B_aGaraDSKWzLes",
6   "github_token": "ghp_iddGlcEgEbq1ti5ciyX5eVKjeyMn1c0KhTeV"
7 }
```

__init__.py

```
1 # core package
```

config.py

```
1 import os

2 import json

3

4

5 class AppConfig:

6     FILE_NAME = "app_config.json"

7

8     @staticmethod

9     def load():

10         default = {

11             "show_welcome": True,

12             "manual_update_only": False,

13             "offline_mode": False,

14             "api_key": "",

15             "github_token": "",

16         }

17         if not os.path.exists(AppConfig.FILE_NAME):

18             return default

19         try:

20             with open(AppConfig.FILE_NAME, "r", encoding="utf-8") as f:

21                 data = json.load(f)

22                 return {**default, **data}

23         except:

24             return default

25

26     @staticmethod

27     def save(data):

28         with open(AppConfig.FILE_NAME, "w", encoding="utf-8") as f:

29             json.dump(data, f, indent=4)

30

31
```

```
32 class RuleManager:
33     FILE_NAME = "rules.json"
34     DEFAULTS = {
35         "large_file_kb": 1000,
36         "todo_weight": 2,
37         "duplicate_weight": 3,
38         "abnormal_weight": 5,
39         "entry_point_penalty": 10,
40     }
41
42     @staticmethod
43     def load():
44         if not os.path.exists(RuleManager.FILE_NAME):
45             return RuleManager.DEFAULTS
46         try:
47             with open(RuleManager.FILE_NAME, "r", encoding="utf-8") as f:
48                 user_rules = json.load(f)
49             return {**RuleManager.DEFAULTS, **user_rules}
50         except:
51             return RuleManager.DEFAULTS
52
53
54 GUIDE_CONTENT = {
55     "Step 1": {
56         "title": "? Buoc 1: Chon file ZIP",
57         "content": "1. Bam nut '?' 1. Chon ZIP'.\n2. Chon file .zip chua source code.\n3. Tool se tu dong quet va phan loai file.",
58     },
59     "Step 2": {
60         "title": "? Buoc 2: Cau hinh bo loc",
61         "content": "Cau hinh Max KB, Bo Folder, Regex de loc bot file rac.\n? Bam '?' LOC LAI' de ap dung.",
62     },
63     "Step 3": {
```

```
64     "title": "? Buoc 3: Chon file",
65     "content": "Chon nhom file ben trai, sau do tích chon file ben phai.\nDung o tim kiem '?' de loc nhanh.",
66 },
67 "Step 4": {
68     "title": "? Buoc 4: Preview",
69     "content": "Click vao file de xem truoc noi dung code (co highlight).",
70 },
71 "Step 5": {
72     "title": "? Buoc 5: Che do xuat",
73     "content": "- Full Source: Xuat toan bo code.\n- Audit Report: Chi xuat thong ke.\n- Code Review: Highlight TODO/FIXME.",
74 },
75 "Step 6": {
76     "title": "? Buoc 6: Xuat file",
77     "content": "Chon dinh dang (PDF/MD) roi bam '? START EXPORT'.\n? Meo: Tích chon 'Nen toi uu' de giam dung luong file PDF.",
78 },
79 "Step 7": {
80     "title": "? Analyze",
81     "content": "Bam '? ANALYZE' de xem thong ke nhanh ve du an va nhan loi khuyen tu AI.",
82 },
83 "Step 8": {
84     "title": "? Nen PDF",
85     "content": "Tinh nang nen se loai bo cac du lieu thua va nen stream noi dung, giup file PDF nhe hon tu 30-60% ma khong mat dinh dang.",
86 },
87 }
```

constants.py

```
1 CURRENT_VERSION = "2.5"
2 UPDATE_URL = "https://gist.githubusercontent.com/Zazaa0606/17573db17b86119dcd926178d6ce2a49/raw/version.json"
```

logger.py

```
1 import sys
2 import os
```

```
3 import datetime
4
5
6 class SafeLogWriter:
7     def __init__(self, mode='stdout'):
8         self.mode = mode
9         self.log_file = "app_debug.log"
10        # Xoa log cu moi lan khoi dong lai app
11        if mode == 'stdout' and os.path.exists(self.log_file):
12            try:
13                os.remove(self.log_file)
14            except:
15                pass
16
17        def write(self, text):
18            if not text:
19                return
20            try:
21                timestamp = datetime.datetime.now().strftime("[%H:%M:%S] ")
22                with open(self.log_file, "a", encoding="utf-8") as f:
23                    if text.strip():
24                        f.write(f"{timestamp}{{self.mode}} {text}\n")
25                    else:
26                        f.write(text)
27            except:
28                pass
29
30        def flush(self):
31            pass
32
33        def isatty(self):
34            return False
```

```
35
36     @property
37     def encoding(self):
38         return 'utf-8'
39
40
41 def setup_logging():
42     """Kich hoạt hệ thống logging, chuyển stdout/stderr sang SafeLogWriter."""
43     sys.stdout = SafeLogWriter('INFO')
44     sys.stderr = SafeLogWriter('ERROR')
45     sys.stdin = None
```

utils.py

```
1 import unicodedata
2
3
4 def clean_text(text):
5     if not text:
6         return ""
7     if not isinstance(text, str):
8         text = str(text)
9     text = unicodedata.normalize('NFKD', text)
10    text = "".join([c for c in text if not unicodedata.combining(c)])
11    text = text.replace("D", "D").replace("d", "d")
12    return text.encode('latin-1', 'replace').decode('latin-1')
```

__init__.py

```
1 # features package
```

__init__.py

```
1 # features/ai_analysis package
```

config.py

```
1 class AIConfig:
2     CATEGORIES = {
3         "Code": {
4             '.py', '.js', '.java', '.cpp', '.c', '.cs', '.ts', '.tsx', '.jsx',
5             '.go', '.rs', '.php', '.rb', '.swift', '.kt',
6         },
7         "Config": {
8             '.json', '.yaml', '.yml', '.xml', '.ini', '.env', '.toml',
9             '.conf', '.config', '.gitignore', '.dockerignore',
10        },
11        "Doc": {'.md', '.txt', '.rst', '.pdf', '.docx', 'LICENSE', 'CHANGELOG'},
12        "Asset": {
13            '.png', '.jpg', '.jpeg', '.gif', '.svg', '.ico',
14            '.mp3', '.wav', '.mp4', '.ttf', '.woff',
15        },
16        "Web": {'.html', '.css', '.scss', '.less', '.sass', '.jsp', '.asp'},
17    }
18
19    PATTERNS = {
20        "TODO": r'(?i)(TODO|FIXME|BUG|HACK|XXX|NOTE):s*(.*)',
21        "ENTRY_POINT": {
22            ".py": r'if __name__ == "__main__":',
23            ".java": r'public static void main',
24            ".c": r'int main',
25            ".cpp": r'int main',
26            ".js": r'(app.listen|ReactDOM.render|server.listen)',
27            ".ts": r'(bootstrap|AppModule)',
28        },
29    }
```

intelligence.py

```
1 import zipfile

2 import re

3 import os

4 import hashlib

5

6 from features.ai_analysis.config import AIConfig

7 from core.config import RuleManager

8

9

10 class ProjectIntelligence:

11     def __init__(self, zip_path, file_entries):

12         self.zip_path = zip_path

13         self.entries = file_entries # List[ZipFileEntry]

14         self.report = {

15             "structure": "Unknown",

16             "categories": {"Code": 0, "Config": 0, "Doc": 0, "Asset": 0, "Web": 0, "Binary/Other": 0},

17             "abnormal_files": [],

18             "duplicates": [],

19             "entry_points": [],

20             "todos": [],

21         }

22

23     def run_analysis(self):

24         self._classify_files()

25         self._detect_structure()

26         self._detect_duplicates_and_abnormal()

27         self._scan_content_semantics()

28         self._calculate_tech_debt_score()

29         return self.report

30

31     def _classify_files(self):
```

```
32     for f in self.entries:
33         found = False
34         for cat, exts in AIConfig.CATEGORIES.items():
35             if f.ext in exts or f.filename in exts:
36                 self.report["categories"][cat] += 1
37                 found = True
38                 break
39         if not found:
40             self.report["categories"]["Binary/Other"] += 1
41
42     def _detect_structure(self):
43         paths = set(f.path for f in self.entries)
44         root_files = {os.path.basename(p) for p in paths if "/" not in p and "\\" not in p}
45
46         if "lerna.json" in root_files or "pnpm-workspace.yaml" in root_files:
47             self.report["structure"] = "Monorepo (Multi-package)"
48         elif any("manage.py" in p for p in paths) and "requirements.txt" in root_files:
49             self.report["structure"] = "Django MVC"
50         elif "composer.json" in root_files and "artisan" in root_files:
51             self.report["structure"] = "Laravel MVC"
52         elif "package.json" in root_files and any("src/App" in p for p in paths):
53             self.report["structure"] = "React/Frontend App"
54         elif "pom.xml" in root_files:
55             self.report["structure"] = "Java Maven Project"
56         elif "go.mod" in root_files:
57             self.report["structure"] = "Go Module"
58         else:
59             self.report["structure"] = "Standard/Generic Project"
60
61     def _detect_duplicates_and_abnormal(self):
62         crc_map = {}
63         with zipfile.ZipFile(self.zip_path, 'r') as zf:
```

```
64         for f in self.entries:
65             if f.size > 1024 * 1024 and f.ext in AIConfig.CATEGORIES["Code"]:
66                 self.report["abnormal_files"].append(
67                     f"{f.path} (Large Code: {f.size/1024:.1f}KB)"
68                 )
69             try:
70                 info = zf.getinfo(f.path)
71                 if info.CRC in crc_map:
72                     self.report["duplicates"].append(f"{f.path} == {crc_map[info.CRC]}")
73             else:
74                 crc_map[info.CRC] = f.path
75             except:
76                 pass
77
78     def _scan_content_semantics(self):
79         scan_limit = 100
80         count = 0
81         with zipfile.ZipFile(self.zip_path, 'r') as zf:
82             for f in self.entries:
83                 if count >= scan_limit:
84                     break
85                 if f.ext not in AIConfig.CATEGORIES["Code"]:
86                     continue
87                 try:
88                     with zf.open(f.path) as file_obj:
89                         content_bytes = file_obj.read(20000)
90                         if b'\0' in content_bytes:
91                             continue
92                         content = content_bytes.decode('utf-8', errors='ignore')
93                         regex = AIConfig.PATTERNS["ENTRY_POINT"].get(f.ext)
94                         if regex and re.search(regex, content):
95                             self.report["entry_points"].append(f.path)
```

```
96         lines = content.split('\n')
97         for i, line in enumerate(lines):
98             m = re.search(AIConfig.PATTERNS["TODO"], line)
99             if m:
100                 clean_msg = m.group(2).strip()[:50]
101                 self.report["todos"].append(
102                     f'[{f.filename}:{i+1}] {m.group(1)}: {clean_msg}'
103                 )
104                 count += 1
105         except:
106             pass
107
108     def generate_advice(self):
109         """
110         Tao goi y thong minh kem theo ly do (Explainability) va do tin cay (Confidence).
111         Return: List of dict {'msg': str, 'reason': str, 'confidence': str}
112         """
113         advice_list = []
114         todos_count = len(self.report["todos"])
115         if todos_count > 5:
116             advice_list.append({
117                 "msg": "Nen chon che do 'Code Review'.",
118                 "reason": f"Phat hien {todos_count} vi tri TODO/FIXME can xu ly.",
119                 "confidence": "Cao",
120             })
121         else:
122             advice_list.append({
123                 "msg": "Nen chon che do 'Full Source' hoac 'Audit Report'.",
124                 "reason": "Code kha sach, it no ky thuat (TODOs).",
125                 "confidence": "Trung binh",
126             })
127
```

```
128     dup_count = len(self.report['duplicates'])
129     if dup_count > 0:
130         advice_list.append({
131             "msg": "Nen tích chọn 'Gop file' hoặc loc bot.",
132             "reason": f"Co {dup_count} file trùng lặp nội dung, gây lãng phí giấy/trang PDF.",
133             "confidence": "Cao",
134         })
135
136     struct = self.report['structure']
137     if struct != "Unknown" and struct != "Standard/Generic Project":
138         advice_list.append({
139             "msg": f"Day la du an {struct}.",
140             "reason": "Phat hien cac file cau hinh dac thu (package.json, pom.xml, etc.).",
141             "confidence": "Tuyet doi",
142         })
143
144     return advice_list
145
146 def _calculate_tech_debt_score(self):
147     rules = RuleManager.load()
148     score = 100
149     causes = []
150
151     todo_count = len(self.report['todos'])
152     if todo_count > 0:
153         weight = rules.get('todo_weight', 2)
154         deduction = min(30, todo_count * weight)
155         score -= deduction
156         causes.append(f"-{deduction} pts: Phat hien {todo_count} vi tri TODO/FIXME (x{weight}).")
157
158     abnormal_count = len(self.report['abnormal_files'])
159     if abnormal_count > 0:
```

```
160         weight = rules.get('abnormal_weight', 5)
161         deduction = min(20, abnormal_count * weight)
162         score -= deduction
163         causes.append(f"-{deduction} pts: Co {abnormal_count} file code kich thuoc bat thuong (x{weight}).")
164
165     dup_count = len(self.report['duplicates'])
166     if dup_count > 0:
167         weight = rules.get('duplicate_weight', 3)
168         deduction = min(20, dup_count * weight)
169         score -= deduction
170         causes.append(f"-{deduction} pts: Co {dup_count} file bi trung lap noi dung (x{weight}).")
171
172     if not self.report['entry_points'] and self.report['categories']['Code'] > 0:
173         penalty = rules.get('entry_point_penalty', 10)
174         score -= penalty
175         causes.append(f"-{penalty} pts: Khong tim thay Entry Point (main/app start) ro rang.")
176
177     score = max(0, score)
178     if score >= 80:
179         risk = "Low (Thap)"
180     elif score >= 50:
181         risk = "Medium (Trung binh)"
182     else:
183         risk = "High (Cao - Can Refactor)"
184
185     self.report["tech_debt"] = {
186         "score": score,
187         "risk_level": risk,
188         "main_causes": causes,
189     }
190
191     def generate_project_tree(self):
```

```
192     """Tao so do cay dang text voi cac icon canh bao"""
193     tree = {}
194     for f in self.entries:
195         parts = f.path.strip("/").split("/")
196         node = tree
197         for part in parts:
198             node = node.setdefault(part, {})
199
200     important_files = {
201         'readme.md', 'package.json', 'requirements.txt', 'docker-compose.yml',
202         'pom.xml', 'build.gradle', 'go.mod', 'cargo.toml', '.env.example',
203     }
204     lines = ["PROJECT ROOT"]
205
206     def _walk(node, prefix=""):
207         keys = sorted(node.keys(), key=lambda x: (len(node[x]) == 0, x.lower()))
208         count = len(keys)
209         for i, key in enumerate(keys):
210             is_last = (i == count - 1)
211             connector = "???" if is_last else "???"
212             icon = ""
213             lower_key = key.lower()
214             is_entry = any(key in p for p in self.report.get('entry_points', []))
215             if is_entry:
216                 icon += " (ENTRY) ?"
217             if lower_key in important_files:
218                 icon += " ??"
219             if "copy" in lower_key or "backup" in lower_key or "old" in lower_key:
220                 icon += " ??"
221             lines.append(f"{prefix}{connector}{key}{icon}")
222             if node[key]:
223                 extension = " " if is_last else "? "
```

```
224         _walk(node[key], prefix + extension)
225
226     _walk(tree)
227     return "\n".join(lines)
228
229     def suggest_reading_order(self):
230         """
231         Goi y thu tu doc file dua tren Heuristic Scoring.
232         Tra ve: List top 5 file quan trong nhat.
233         """
234         candidates = []
235         high_value_keywords = ['core', 'main', 'app', 'server', 'index', 'manage', 'settings', 'config', 'model', 'service']
236         low_value_keywords = ['test', 'spec', 'mock', 'min', 'backup', 'old', 'temp']
237
238         for f in self.entries:
239             score = 10
240             reasons = []
241             name_lower = f.filename.lower()
242             path_lower = f.path.lower()
243
244             if name_lower.startswith('readme'):
245                 score += 500
246                 reasons.append("Tai lieu du an (Must Read)")
247             elif name_lower.startswith('guide') or name_lower.startswith('install'):
248                 score += 200
249                 reasons.append("Huong dan su dung")
250
251             if f.path in self.report.get('entry_points', []):
252                 score += 300
253                 reasons.append("Entry Point (Diem khoi chay)")
254
255             if name_lower in ['package.json', 'requirements.txt', 'pom.xml', 'build.gradle', 'go.mod', 'docker-compose.yml']:
```

```
256         score += 150
257         reasons.append("Cau hinh & Thu vien")
258
259     if any(kw in name_lower for kw in high_value_keywords):
260         score += 50
261
262     depth = f.path.count('/')
263     if depth == 0:
264         score += 30
265
266     if any(kw in path_lower for kw in low_value_keywords):
267         score -= 200
268
269     if score > 20 and f.ext in (
270         AIConfig.CATEGORIES['Code']
271         .union(AIConfig.CATEGORIES['Doc'])
272         .union(AIConfig.CATEGORIES['Config'])
273     ):
274         candidates.append({
275             "path": f.path,
276             "score": score,
277             "reason": ", ".join(reasons) if reasons else "File quan trong o thu muc goc",
278         })
279
280     candidates.sort(key=lambda x: x['score'], reverse=True)
281     return candidates[:5]
282
283 def calculate_fingerprint(self):
284     """
285     Tao 'Dau van tay' duy nhat cho du an dua tren tong so file, CRC32, va ten file.
286     """
287     try:
```

```
288     data_points = []
289     with zipfile.ZipFile(self.zip_path, 'r') as zf:
290         sorted_entries = sorted(self.entries, key=lambda x: x.path)
291         for entry in sorted_entries:
292             try:
293                 info = zf.getinfo(entry.path)
294                 data_points.append(f"{entry.path}{{info.CRC}}{info.file_size}")
295             except:
296                 pass
297     raw_string = ";".join(data_points)
298     full_hash = hashlib.sha256(raw_string.encode('utf-8')).hexdigest().upper()
299     short_hash = f"{full_hash[:4]}-{{full_hash[4:8]}}-{{full_hash[8:12]}}-{{full_hash[12:16]}}"
300     return short_hash, full_hash
301 except Exception as e:
302     return "ERROR-CALC-HASH", str(e)
```

__init__.py

```
1 # features/github_import package
```

__init__.py

```
1 # features/pdf_export package
```

base.py

```
1 from abc import ABC, abstractmethod
2
3
4 class BaseExporter(ABC):
5     @abstractmethod
6     def export(self, zip_path, file_paths, save_path, options, progress_callback, cancel_event):
7         pass
8
```

```
9
10 class ExporterRegistry:
11     _exporters = {}
12
13     @classmethod
14     def register(cls, name, extension):
15         def decorator(subclass):
16             cls._exporters[name] = {"class": subclass, "ext": extension}
17             return subclass
18         return decorator
19
20     @classmethod
21     def get_all(cls):
22         return cls._exporters
```

markdown_exporter.py

```
1 import zipfile
2 import os
3
4 from features.pdf_export.base import BaseExporter, ExporterRegistry
5
6
7 @ExporterRegistry.register("Markdown", ".md")
8 class MarkdownExporter(BaseExporter):
9     def export(self, zip_path, file_paths, save_path, options, progress_callback, cancel_event):
10         total = len(file_paths)
11         with open(save_path, 'w', encoding='utf-8') as out_f:
12             out_f.write("# Project Export\n\n")
13             with zipfile.ZipFile(zip_path, 'r') as zf:
14                 for idx, p in enumerate(file_paths):
15                     if cancel_event.is_set():
16                         return
```

```
17         progress_callback(idx + 1, total, os.path.basename(p))
18         ext = os.path.splitext(p)[1].replace(':', '')
19         out_f.write(f'## File: {p}\n```${ext}\n")
20     try:
21         out_f.write(zf.read(p).decode('utf-8', 'ignore'))
22     except:
23         out_f.write("Error reading file.")
24     out_f.write(f"\n```\n\n")
```

pdf_exporter.py

```
1  import zipfile
2
3  import os
4
5  import re
6
7  import io
8
9  import datetime
10
11
12
13  from pypdf import PdfReader, PdfWriter
14
15  from fpdf import FPDF
16
17  from pygments import lex
18
19  from pygments.lexers import get_lexer_for_filename, TextLexer
20
21  from pygments.token import Keyword, Name, Comment, String, Number, Operator, Punctuation
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

```
23     self.watermark_text = watermark_text
24
25     self.current_file = ""
26
27     self.theme = {
28         Keyword: (0, 0, 255),
29         Name.Function: (122, 122, 0),
30         Comment: (0, 128, 0),
31         String: (163, 21, 21),
32         Number: (9, 134, 115),
33         Operator: (0, 0, 0),
34         Punctuation: (0, 0, 0),
35     }
36
37     def header(self):
38         if self.watermark_text:
39             try:
40                 self.set_font("Arial", 'B', 50)
41                 self.set_text_color(240, 240, 240)
42                 w = self.get_string_width(self.watermark_text)
43                 x_pos = (210 - w) / 2
44                 y_pos = 140
45                 self.text(x_pos, y_pos, self.watermark_text)
46             except:
47                 pass
48
49             self.set_text_color(128)
50
51             self.set_font(self.main_font_name, "", 8)
52             self.set_text_color(128)
53
54             safe_zip_name = clean_text(self.zip_name)
55
56             disp_zip = (safe_zip_name[:40] + '...') if len(safe_zip_name) > 40 else safe_zip_name
57
58             self.cell(0, 10, f"Project: {disp_zip}", 0, 0, 'L')
59
60             self.set_x(10)
61
62         if self.current_file:
```

```
55         safe_filename = clean_text(self.current_file)
56         disp_file = ('...' + safe_filename[-50:]) if len(safe_filename) > 50 else safe_filename
57         self.cell(0, 10, f"File: {disp_file}", 0, 0, 'R')
58     self.ln(12)
59
60     def footer(self):
61         self.set_y(-15)
62         self.set_font(self.main_font_name, "", 8)
63         self.set_text_color(128)
64         self.cell(0, 10, f"Page {self.page_no()} / {{nb}}", 0, 0, 'C')
65
66     def draw_summary_block(self, title, content, x, y, w, h, color=(240, 240, 240)):
67         """Ve mot khoi thong tin cho trang Summary"""
68         self.set_fill_color(*color)
69         self.rect(x, y, w, h, 'F')
70         self.set_xy(x + 2, y + 2)
71         self.set_font(self.main_font_name, 'B', 10)
72         self.set_text_color(0)
73         self.cell(w - 4, 6, clean_text(title), ln=True)
74         self.set_xy(x + 2, y + 10)
75         self.set_font(self.main_font_name, "", 9)
76         self.multi_cell(w - 4, 5, clean_text(content))
77
78
79     @ExporterRegistry.register("PDF Document", ".pdf")
80     class PDFExporter(BaseExporter):
81         def __init__(self):
82             self.lexer_cache = {}
83             self.bookmarks = []
84
85         def _get_cached_lexer(self, filename):
86             ext = os.path.splitext(filename)[1].lower()
```

```
87     if ext in self.lexer_cache:
88         return self.lexer_cache[ext]
89     try:
90         lexer = get_lexer_for_filename(filename)
91     except:
92         lexer = TextLexer()
93     self.lexer_cache[ext] = lexer
94     return lexer
95
96 def export(self, zip_path, file_paths, save_path, options, progress_callback, cancel_event):
97     total = len(file_paths)
98     font_name = "Arial"
99
100    mode = options.get('mode', 'Full Source Code')
101    is_struct_only = options.get('struct_only', False)
102    w_text = options.get('watermark', '')
103    zip_name = os.path.basename(zip_path)
104
105    pdf = IDE_Style_PDF(font_name, zip_name=zip_name, watermark_text=w_text)
106    pdf.set_title("Project Structure Report" if is_struct_only else "Project Scanner Report")
107    pdf.set_author("Zip2Pdf Pro AI")
108    pdf.alias_nb_pages()
109
110    if is_struct_only:
111        pdf.add_page()
112        pdf.set_font("Arial", "B", 20)
113        pdf.cell(0, 10, "PROJECT STRUCTURE MAP", ln=True, align="C")
114        pdf.ln(5)
115        pdf.set_font("Arial", "", 10)
116        pdf.cell(0, 10, f"Source: {zip_name}", ln=True, align="C")
117        pdf.ln(10)
118        tree_lines = self._generate_tree_text(file_paths)
```

```
119         pdf.set_font("Courier", "", 10)
120         line_height = 5
121         for line in tree_lines:
122             if pdf.get_y() > 270:
123                 pdf.add_page()
124                 pdf.set_font("Courier", "", 10)
125                 if "?" in line:
126                     pdf.set_text_color(0, 0, 139)
127                     pdf.set_font("Courier", "B", 10)
128                 else:
129                     pdf.set_text_color(0)
130                     pdf.set_font("Courier", "", 10)
131                 safe_line = clean_text(line)
132                 pdf.cell(0, line_height, safe_line, ln=True)
133         pdf.output(save_path)
134         return
135
136     self._add_executive_summary(pdf, zip_name, mode, options.get('ai_report', {}), len(file_paths))
137
138     curr_dir = ""
139     with zipfile.ZipFile(zip_path, 'r') as zf:
140         for idx, p in enumerate(file_paths):
141             if cancel_event.is_set():
142                 return
143             progress_callback(idx + 1, total, os.path.basename(p))
144
145             pdf.current_file = p
146             d = os.path.dirname(p)
147
148             if d != curr_dir:
149                 self.bookmarks.append({'title': f"? {d if d else 'Root'}", "page": pdf.page_no() - 1, "level": 0})
150                 if options.get('section'):
```

```
151         pdf.add_page()
152         pdf.set_fill_color(50, 50, 50)
153         pdf.set_text_color(255)
154         pdf.set_font(font_name, "B", 14)
155         pdf.cell(0, 12, f" MODULE: {d}", fill=True, ln=True)
156         pdf.set_text_color(0)
157         pdf.ln(5)
158         curr_dir = d
159
160         is_merge = options.get('merge')
161         if options.get('one_page') or (is_merge and pdf.get_y() > 250):
162             pdf.add_page()
163         elif is_merge:
164             pdf.ln(5)
165             pdf.set_draw_color(200)
166             pdf.line(10, pdf.get_y(), 200, pdf.get_y())
167             pdf.ln(5)
168         else:
169             pdf.add_page()
170
171         self.bookmarks.append({"title": f"? {os.path.basename(p)}", "page": pdf.page_no() - 1, "level": 1})
172
173         pdf.set_font(font_name, "B", 10)
174         pdf.set_fill_color(230, 240, 250)
175         pdf.set_draw_color(0)
176         pdf.cell(0, 7, f" {os.path.basename(p)}", fill=True, ln=True, border=1)
177         pdf.ln(1)
178
179         try:
180             with zf.open(p) as f:
181                 if b'\0' in f.read(1024):
182                     pdf.set_text_color(128)
```

```
183         pdf.cell(0, 5, "[Binary File - Skipped]", ln=True)
184         continue
185         f.seek(0)
186         text_stream = io.TextIOWrapper(f, encoding='utf-8', errors='replace')
187         self._render_code_stream(pdf, p, text_stream, font_name, options.get('line_num'), mode)
188         text_stream.detach()
189         except Exception as e:
190             pdf.set_text_color(255, 0, 0)
191             pdf.cell(0, 5, f"Error: {str(e)}", ln=True)
192
193 pdf.output(save_path)
194 self._inject_bookmarks_and_compress(
195     save_path,
196     compress=options.get('compress', False),
197     password=options.get('password', ''),
198 )
199
200 def _generate_tree_text(self, file_paths):
201     """Ham tao danh sach cac dong text bieu dien cay thu muc (Dang ASCII an toan)"""
202     tree = {}
203     for path in file_paths:
204         parts = path.strip("/").split("/")
205         node = tree
206         for part in parts:
207             node = node.setdefault(part, {})
208     lines = []
209
210     def _walk(node, prefix=""):
211         keys = sorted(node.keys(), key=lambda x: (len(node[x]) == 0, x.lower()))
212         count = len(keys)
213         for i, key in enumerate(keys):
214             is_last = (i == count - 1)
```

```
215         connector = "+-- " if is_last else "|-- "
216         icon = "[DIR] " if node[key] else "  "
217         lines.append(f"{prefix}{connector}{icon}{key}")
218         if node[key]:
219             extension = "  " if is_last else "| "
220             _walk(node[key], prefix + extension)
221
222     _walk(tree)
223     return lines
224
225 def _add_executive_summary(self, pdf, zip_name, mode, report, file_count):
226     """Tao trang bìa báo cáo quan tri"""
227     pdf.add_page()
228     pdf.set_font("Arial", "B", 24)
229     pdf.cell(0, 15, "EXECUTIVE SUMMARY", ln=True, align="C")
230     pdf.set_font("Arial", "", 12)
231     pdf.cell(0, 8, f"Project: {zip_name}", ln=True, align="C")
232     pdf.ln(10)
233
234     tech_debt = report.get("tech_debt", {"score": 100, "risk_level": "Unknown"})
235     score = tech_debt["score"]
236
237     if score >= 80:
238         color = (200, 255, 200)
239     elif score >= 50:
240         color = (255, 240, 200)
241     else:
242         color = (255, 220, 220)
243
244     pdf.set_fill_color(*color)
245     pdf.rect(10, pdf.get_y(), 190, 30, 'F')
246     pdf.set_y(pdf.get_y() + 5)
```

```
247     pdf.set_font("Arial", "B", 14)
248     risk_info = f" Technical Debt Score: {score}/100 ({tech_debt['risk_level']})"
249     pdf.cell(0, 10, clean_text(risk_info), ln=True)
250
251     fingerprint = report.get("fingerprint", "N/A")
252     pdf.set_font("Arial", "B", 10)
253     pdf.set_text_color(100, 100, 100)
254     text_fp = f" Project Fingerprint: {fingerprint} [ID]"
255     pdf.cell(0, 6, clean_text(text_fp), ln=True)
256     pdf.set_text_color(0)
257
258     pdf.set_font("Arial", "I", 10)
259     pdf.cell(0, 8, f" Generated by Zip2Pdf Pro AI at {datetime.datetime.now().strftime('%Y-%m-%d %H:%M')}", ln=True)
260     pdf.ln(15)
261
262     advice_text = "No specific advice."
263     if report.get('todos'):
264         advice_text = f"Found {len(report['todos'])} TODOs. Consider 'Code Review' mode."
265     elif report.get('duplicates'):
266         advice_text = f"Found {len(report['duplicates'])} duplicates. Consider cleaning up."
267     else:
268         advice_text = "Project looks clean. Good job!"
269
270     pdf.draw_summary_block("AI QUICK ADVICE", advice_text, 10, pdf.get_y(), 190, 30)
271     pdf.add_page()
272
273     def _render_code_stream(self, pdf, filename, text_stream, font_name, show_line_num, mode):
274         lexer = self._get_cached_lexer(filename)
275         pdf.set_font(font_name, "", 8)
276         line_idx = 1
277         todo_pattern = re.compile(r'(TODO|FIXME|BUG|HACK|XXX)', re.IGNORECASE)
278
```

```
279     for txt in text_stream:
280         txt = txt.rstrip("\n")
281         if pdf.get_y() > 275:
282             pdf.add_page()
283
284         if mode == "Code Review" and todo_pattern.search(txt):
285             pdf.ln(1)
286             pdf.set_fill_color(255, 235, 235)
287             pdf.set_draw_color(200, 0, 0)
288             pdf.set_font(font_name, "B", 8)
289             pdf.set_text_color(200, 0, 0)
290             pdf.cell(0, 5, f" REVIEW NOTE (Line {line_idx}): Potential Tech Debt", ln=True, fill=True, border='LTR')
291             pdf.set_font(font_name, "", 8)
292             pdf.set_text_color(0)
293             safe_txt = clean_text(txt).strip()
294             pdf.multi_cell(0, 5, f" {safe_txt}", border='LBR', fill=True)
295             pdf.set_draw_color(0)
296             pdf.ln(1)
297             line_idx += 1
298             continue
299
300         if show_line_num:
301             pdf.set_text_color(160)
302             pdf.set_font_size(7)
303             pdf.cell(8, 4, str(line_idx), align='R')
304
305             pdf.set_font_size(8)
306
307         if len(txt) > 500:
308             pdf.set_text_color(0)
309             pdf.set_font(font_name, "", 8)
310             display_txt = clean_text(txt[:2000] + "... if len(txt) > 2000 else txt)
```

```
311         pdf.write(4, display_txt)
312     else:
313         for t, v in lex(txt, lexer):
314             c = pdf.theme.get(t) or pdf.theme.get(t.parent) or (0, 0, 0)
315             safe_v = clean_text(v)
316             pdf.set_text_color(*c)
317             pdf.set_font(font_name, "", 8)
318             pdf.write(4, safe_v)
319
320     pdf.ln()
321     line_idx += 1
322
323 def _inject_bookmarks_and_compress(self, file_path, compress=False, password=""):
324     """Su dung pypdf de them Bookmark that, nen file va dat mat khau"""
325     try:
326         reader = PdfReader(file_path)
327         writer = PdfWriter()
328         writer.append_pages_from_reader(reader)
329
330         parent_map = {}
331         for b in self.bookmarks:
332             if b['level'] == 0:
333                 parent = writer.add_outline_item(b['title'], b['page'])
334                 parent_map[0] = parent
335             else:
336                 parent_node = parent_map.get(0, None)
337                 writer.add_outline_item(b['title'], b['page'], parent=parent_node)
338
339         if compress:
340             for page in writer.pages:
341                 page.compress_content_streams()
342
```

```
343         if password:
344             writer.encrypt(password)
345
346         with open(file_path, "wb") as f:
347             writer.write(f)
348
349     except Exception as e:
350         print(f"Post-processing error: {e}")
```

__init__.py

```
1 # features/pdf_to_zip package
```

converter.py

```
1 import zipfile
2 import os
3 import re
4 import threading
5
6
7 class PdfToZipConverter:
8     def __init__(self):
9         self.stop_event = threading.Event()
10
11     def convert(self, pdf_path, output_zip_path, progress_callback):
12         try:
13             from pypdf import PdfReader
14             reader = PdfReader(pdf_path)
15             if not reader.outline:
16                 return False, "PDF này không có Bookmark (Mục lục). Không thể tạo cấu trúc folder."
17
18             file_map = {}
19
```

```
20     def efficient_parse(items, parent_path=""):
21         i = 0
22         while i < len(items):
23             item = items[i]
24             if isinstance(item, list):
25                 prev_item = items[i - 1] if i > 0 else None
26                 new_base = parent_path
27                 if prev_item and not isinstance(prev_item, list):
28                     t = prev_item.title
29                     if "?" in t:
30                         folder_name = t.replace("?", "").strip()
31                         new_base = os.path.join(parent_path, folder_name)
32                 efficient_parse(item, new_base)
33                 i += 1
34                 continue
35             title = item.title
36             if "?" in title:
37                 fname = title.replace("?", "").strip()
38                 full_path = os.path.join(parent_path, fname).replace("\\", "/")
39             try:
40                 idx = reader.get_destination_page_number(item)
41                 file_map[idx] = full_path
42             except:
43                 pass
44             i += 1
45
46     efficient_parse(reader.outline, "")
47
48     if not file_map:
49         return False, "Không tìm thấy cấu trúc cây (Bookmarks hop le) trong PDF."
50
51     sorted_starts = sorted(file_map.keys())
```

```
52     total_pages = len(reader.pages)
53
54     line_num_pattern = re.compile(r'^\s*(\d+)\s+')
55
56     with zipfile.ZipFile(output_zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
57         for i, start_page in enumerate(sorted_starts):
58             if self.stop_event.is_set():
59                 break
60
61             full_path = file_map[start_page]
62             end_page = sorted_starts[i + 1] if i + 1 < len(sorted_starts) else total_pages
63             file_content_lines = []
64
65             for p_idx in range(start_page, end_page):
66                 progress_callback(p_idx, total_pages, f'Restoring: {full_path}')
67                 text = reader.pages[p_idx].extract_text(extraction_mode="layout")
68                 if not text:
69                     continue
70                 lines = text.split("\n")
71                 for line in lines:
72                     if "Page" in line and "/" in line and len(line) < 20:
73                         continue
74                     if "Project:" in line and "File:" in line:
75                         continue
76                     if "REVIEW NOTE" in line:
77                         continue
78                     if "Generated by Zip2Pdf" in line:
79                         continue
80                     if "EXECUTIVE SUMMARY" in line:
81                         continue
82                 cleaned = line_num_pattern.sub("", line, count=1)
83                 file_content_lines.append(cleaned.rstrip())
```

```
84
85         full_text_content = "\n".join(file_content_lines)
86         zf.writestr(full_path, full_text_content.encode('utf-8'))
87
88     return True, output_zip_path
89
90 except Exception as e:
91     return False, str(e)
```

__init__.py

```
1 # features/workspace package
```

manager.py

```
1 import json
2 import os
3
4
5 class WorkspaceManager:
6     FILE_NAME = "workspaces.json"
7
8     @staticmethod
9     def load_workspaces():
10         if not os.path.exists(WorkspaceManager.FILE_NAME):
11             return {}
12         try:
13             with open(WorkspaceManager.FILE_NAME, "r", encoding="utf-8") as f:
14                 return json.load(f)
15         except:
16             return {}
17
18     @staticmethod
19     def save_workspace(name, app_state):
```

```
20     ws = WorkspaceManager.load_workspaces()
21     ws[name] = app_state
22     with open(WorkspaceManager.FILE_NAME, "w", encoding="utf-8") as f:
23         json.dump(ws, f, indent=4)
24
25
26 class PresetManager:
27     FILE_NAME = "presets.json"
28
29     @staticmethod
30     def load_presets():
31         if not os.path.exists(PresetManager.FILE_NAME):
32             return {}
33         try:
34             with open(PresetManager.FILE_NAME, "r", encoding="utf-8") as f:
35                 return json.load(f)
36         except:
37             return {}
38
39     @staticmethod
40     def save_preset(name, data):
41         presets = PresetManager.load_presets()
42         presets[name] = data
43         with open(PresetManager.FILE_NAME, "w", encoding="utf-8") as f:
44             json.dump(presets, f, indent=4)
45
46     @staticmethod
47     def delete_preset(name):
48         presets = PresetManager.load_presets()
49         if name in presets:
50             del presets[name]
51         with open(PresetManager.FILE_NAME, "w", encoding="utf-8") as f:
```

```
52         json.dump(presets, f, indent=4)
```

__init__.py

```
1 # features/zip_scanner package
```

scanner.py

```
1 import zipfile
2 import os
3 import re
4 import tempfile
5 import time
6
7 from models.zip_entry import ZipFileEntry
8
9
10 class ZipScanner:
11     def __init__(self):
12         self.zip_path = ""
13         self.all_files = []
14         self.last_report = {}
15         self.valid_exts = {
16             '.txt', '.py', '.js', '.html', '.css', '.java', '.cpp', '.c',
17             '.md', '.json', '.xml', '.csv', '.sql', '.ts', '.tsx', '.jsx',
18             '.scss', '.yaml', '.sh', '.bat', '.env', '.config', '.ini',
19             '.ps1', '.php', '.go', '.rs', '.kt', '.kts', '.gradle', '.properties',
20         }
21         self.all_detected_exts = set()
22         self.selected_exts = set()
23
24     def load_zip(self, path):
25         self.zip_path = path
26         self.all_files = []
```

```
27     try:
28         with zipfile.ZipFile(path, 'r') as zf:
29             for i in zf.infolist():
30                 if not i.is_dir():
31                     self.all_files.append(ZipFileEntry(i.filename, i.file_size))
32             return True, len(self.all_files)
33     except Exception as e:
34         return False, str(e)
35
36 def filter_files(self, max_kb, ignore_str, regex_str, allowed_exts=None):
37     try:
38         max_b = float(max_kb) * 1024
39     except:
40         max_b = 50000
41     ignores = [x.strip() for x in ignore_str.split(',') if x.strip()]
42     reg = regex_str.strip()
43     filtered_list = []
44     grouped_map = {}
45
46     for f in self.all_files:
47         if f.size > max_b:
48             continue
49         if allowed_exts is not None:
50             if f.ext not in allowed_exts:
51                 continue
52         elif f.ext not in self.valid_exts:
53             continue
54         if any(bad in f.path for bad in ignores):
55             continue
56         if reg and not re.search(reg, f.filename):
57             continue
58         filtered_list.append(f)
```

```
59         grouped_map.setdefault(f.ext, []).append(f)
60
61     return filtered_list, grouped_map
62
63     def read_file_preview(self, file_path, max_chars=2000):
64         try:
65             with zipfile.ZipFile(self.zip_path) as z:
66                 info = z.getinfo(file_path)
67                 with z.open(file_path) as f:
68                     if b'\0' in f.read(1024):
69                         return "[Binary File - Preview Unavailable]"
70                     if info.file_size > 50 * 1024:
71                         with z.open(file_path) as f:
72                             return f.read(2000).decode('utf-8', 'ignore') + "\n\n... [PREVIEW TRUNCATED] ..."
73                     with z.open(file_path) as f:
74                         return f.read(max_chars).decode('utf-8', 'ignore')
75         except:
76             return "Error reading preview."
77
78     def analyze_selection(self, selected_paths):
79         from features.ai_analysis.intelligence import ProjectIntelligence
80
81         selected_entries = [f for f in self.all_files if f.path in selected_paths]
82         if not selected_entries:
83             return False, "No files selected."
84         try:
85             ai = ProjectIntelligence(self.zip_path, selected_entries)
86             report = ai.run_analysis()
87             self.last_report = report
88             stats = {
89                 "total_files": len(selected_entries),
90                 "total_size": sum(f.size for f in selected_entries),
```

```
91         "ai_report": report,
92     }
93     return True, stats
94 except Exception as e:
95     return False, str(e)
96
97 def create_virtual_zip(self, source_type, data, ignore_str=""):
98     """
99     Tao file ZIP tam thoi co TOI UU HOA bo qua folder rac (node_modules, .git...)
100     """
101     try:
102         tmp_dir = tempfile.mkdtemp()
103         zip_path = os.path.join(tmp_dir, "virtual_project.zip")
104
105         hard_blocks = {
106             'node_modules', '.git', '.svn', '.hg', '.idea', '.vscode',
107             '__pycache__', 'venv', 'env', 'dist', 'build', 'target', 'bin', 'obj',
108         }
109         user_ignores = {x.strip().lower() for x in ignore_str.split(',') if x.strip()}
110         final_ignores = hard_blocks.union(user_ignores)
111
112         with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
113             if source_type == "folder":
114                 base_len = len(data) + 1
115                 for root, dirs, files in os.walk(data):
116                     dirs[:] = [d for d in dirs if d.lower() not in final_ignores]
117                     for f in files:
118                         if f.lower().endswith(('log', 'tmp', 'lock')):
119                             continue
120                         fp = os.path.join(root, f)
121                         arcname = fp[base_len:]
122                         try:
```

```
123             zf.write(fp, arcname)
124         except:
125             pass
126     elif source_type == "files":
127         for fp in data:
128             zf.write(fp, os.path.basename(fp))
129     elif source_type == "text":
130         fname, content = data
131         zf.writestr(fname, content)
132
133     return True, zip_path
134 except Exception as e:
135     return False, str(e)
136
137 def create_virtual_multisource_zip(self, inputs):
138     """
139     Gop nhieu nguon (Path file zip, Path folder) thanh 1 Zip tong hop duy nhat.
140     inputs = list cac tuple: [("zip", "path/to/A.zip"), ("folder", "path/to/B")]
141     """
142     try:
143         tmp_dir = tempfile.mkdtemp()
144         master_zip_path = os.path.join(tmp_dir, "MasterProject_Merged.zip")
145
146         with zipfile.ZipFile(master_zip_path, 'w', zipfile.ZIP_DEFLATED) as master_zf:
147             for dtype, dpath in inputs:
148                 root_name = os.path.basename(dpath.rstrip("/\\"))
149                 if not root_name:
150                     root_name = f"Project_{int(time.time())}"
151
152                 if dtype == "zip":
153                     try:
154                         with zipfile.ZipFile(dpath, 'r') as src_zf:
```

```
155         for item in src_zf.infolist():
156             if item.is_dir():
157                 continue
158                 new_arcname = f"{root_name}/{item.filename}"
159                 master_zf.writestr(new_arcname, src_zf.read(item.filename))
160         except:
161             pass
162         elif dtype == "folder":
163             base_len = len(dpath) + 1
164             for root, _, files in os.walk(dpath):
165                 for f in files:
166                     full_fp = os.path.join(root, f)
167                     rel_path = full_fp[base_len:]
168                     new_arcname = os.path.join(root_name, rel_path).replace("\\", "/")
169                     master_zf.write(full_fp, new_arcname)
170
171         return True, master_zip_path
172     except Exception as e:
173         return False, str(e)
```

__init__.py

```
1 # models package
```

selection.py

```
1 class SelectionModel:
2     def __init__(self):
3         self.selected_paths = set()
4         self.history = [] # Stack cho Undo
5         self.redo_stack = [] # Stack cho Redo
6         self._save_state() # Lưu trạng thái ban đầu
7
8     def _save_state(self):
```

```
9     self.history.append(set(self.selected_paths))

10     if len(self.history) > 20:

11         self.history.pop(0)

12     self.redo_stack.clear()

13

14     def undo(self):

15         if len(self.history) > 1:

16             self.redo_stack.append(self.history.pop())

17             self.selected_paths = set(self.history[-1])

18             return True

19         return False

20

21     def redo(self):

22         if self.redo_stack:

23             state = self.redo_stack.pop()

24             self.history.append(state)

25             self.selected_paths = set(state)

26             return True

27         return False

28

29     def add(self, path):

30         self._save_state()

31         self.selected_paths.add(path)

32

33     def remove(self, path):

34         self._save_state()

35         self.selected_paths.discard(path)

36

37     def toggle(self, path):

38         self._save_state()

39         if path in self.selected_paths:

40             self.selected_paths.discard(path)
```

```
41     else:
42         self.selected_paths.add(path)
43
44     def set_all(self, items):
45         self._save_state()
46         for item in items:
47             self.selected_paths.add(item.path)
48
49     def clear_all(self, items):
50         self._save_state()
51         for item in items:
52             self.selected_paths.discard(item.path)
53
54     def clear_set(self):
55         self._save_state()
56         self.selected_paths.clear()
57
58     def is_selected(self, path):
59         return path in self.selected_paths
60
61     def get_list(self):
62         return sorted(list(self.selected_paths))
63
64     def is_empty(self):
65         return len(self.selected_paths) == 0
```

zip_entry.py

```
1 import os
2
3
4 class ZipFileEntry:
5     __slots__ = ['path', 'size', 'ext', 'filename']
```

```
6
7  def __init__(self, path, size):
8      self.path = path
9      self.size = size
10     self.ext = os.path.splitext(path)[1].lower()
11     self.filename = os.path.basename(path)
```

`__init__.py`

```
1 # ui package
```

`__init__.py`

```
1 # ui/components package
```

`code_preview.py`

```
1 import tkinter as tk
2 from tkinter import ttk
3 from pygments import lex
4 from pygments.lexers import get_lexer_for_filename, TextLexer
5 from pygments.token import Keyword, Name, Comment, String
6
7
8 class CodePreviewWidget(tk.Frame):
9     def __init__(self, parent, **kwargs):
10         super().__init__(parent, **kwargs)
11         self.v_scroll = ttk.Scrollbar(self, orient="vertical", command=self._on_scroll)
12         self.v_scroll.pack(side=tk.RIGHT, fill=tk.Y)
13         self.line_nums = tk.Text(
14             self, width=4, padx=4, pady=0, takefocus=0, border=0,
15             background="#f0f0f0", foreground="#888888", state="disabled",
16             font=("Consolas", 9),
17         )
```

```
18     self.line_nums.pack(side=tk.LEFT, fill=tk.Y)
19     self.text_area = tk.Text(
20         self, wrap="none", padx=4, pady=0, border=0,
21         font=("Consolas", 9), background="white", undo=False,
22     )
23     self.text_area.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
24     self.text_area.config(yscrollcommand=self._update_scroll)
25     self.line_nums.config(yscrollcommand=self._null_scroll)
26     self._setup_tags()
27
28     def _setup_tags(self):
29         style = {
30             'Keyword': '#0000ff',
31             'Name.Function': '#795e26',
32             'Name.Class': '#267f99',
33             'String': '#a31515',
34             'Comment': '#008000',
35             'Number': '#098658',
36             'Operator': '#333333',
37             'Punctuation': '#333333',
38         }
39         for token_name, color in style.items():
40             self.text_area.tag_config(token_name, foreground=color)
41
42     def _on_scroll(self, *args):
43         self.text_area.yview(*args)
44         self.line_nums.yview(*args)
45
46     def _update_scroll(self, first, last):
47         self.text_area.yview_moveto(first)
48         self.line_nums.yview_moveto(first)
49         self.v_scroll.set(first, last)
```

```
50
51 def _null_scroll(self, *args):
52     pass
53
54 def set_content(self, filename, content):
55     self.text_area.config(state="normal")
56     self.line_nums.config(state="normal")
57     self.text_area.delete("1.0", tk.END)
58     self.line_nums.delete("1.0", tk.END)
59     try:
60         lexer = get_lexer_for_filename(filename)
61     except:
62         lexer = TextLexer()
63     for token, text in lex(content, lexer):
64         tag = str(token)
65         if token in Keyword:
66             tag = 'Keyword'
67         elif token in Name.Function:
68             tag = 'Name.Function'
69         elif token in String:
70             tag = 'String'
71         elif token in Comment:
72             tag = 'Comment'
73         self.text_area.insert(
74             tk.END, text,
75             tag if tag in ['Keyword', 'Name.Function', 'String', 'Comment'] else None,
76         )
77     line_count = int(self.text_area.index('end-1c').split('.')[0])
78     self.line_nums.insert("1.0", "\n".join(str(i) for i in range(1, line_count + 1)))
79     self.text_area.config(state="disabled")
80     self.line_nums.config(state="disabled")
```

tooltip.py

```
1 import tkinter as tk
2 from tkinter import ttk
3
4
5 class ToolTip:
6     def __init__(self, widget, text):
7         self.widget = widget
8         self.text = text
9         self.tip_window = None
10        self.widget.bind("<Enter>", self.show_tip)
11        self.widget.bind("<Leave>", self.hide_tip)
12
13    def show_tip(self, event=None):
14        if self.tip_window or not self.text:
15            return
16        try:
17            x, y, cx, cy = self.widget.bbox("insert")
18        except:
19            x, y = 0, 0
20        x = x + self.widget.winfo_rootx() + 25
21        y = y + self.widget.winfo_rooty() + 25
22        self.tip_window = tw = tk.Toplevel(self.widget)
23        tw.wm_overrideredirect(True)
24        tw.wm_geometry(f"+{x}+{y}")
25        label = tk.Label(
26            tw, text=self.text, justify=tk.LEFT,
27            background="#ffffe0", relief=tk.SOLID, borderwidth=1,
28            font=("tahoma", "8", "normal"),
29        )
30        label.pack(ipadx=1)
31
```

```
32     def hide_tip(self, event=None):
33         if self.tip_window:
34             self.tip_window.destroy()
35             self.tip_window = None
```

theme.py

```
1  import tkinter as tk
2  from tkinter import ttk
3
4
5  class ThemeManager:
6      PALETTES = {
7          "light": {
8              "bg": "#f0f0f0", "fg": "black", "text_bg": "white", "text_fg": "black",
9              "select_bg": "#0078d7", "select_fg": "white", "tree_bg": "white",
10         },
11         "dark": {
12             "bg": "#2b2b2b", "fg": "#e0e0e0", "text_bg": "#1e1e1e", "text_fg": "#d4d4d4",
13             "select_bg": "#4a4a4a", "select_fg": "white", "tree_bg": "#333333",
14         },
15         "matrix": {
16             "bg": "#000000", "fg": "#00FF41", "text_bg": "#0D0208", "text_fg": "#00FF41",
17             "select_bg": "#003B00", "select_fg": "#FFFFFF", "tree_bg": "#000000",
18         },
19     }
20
21     @staticmethod
22     def apply_theme(root, mode="light"):
23         p = ThemeManager.PALETTES[mode]
24         style = ttk.Style(root)
25         style.theme_use('clam')
26         style.configure(".", background=p["bg"], foreground=p["fg"], fieldbackground=p["text_bg"])
```

```
27     style.configure("TLabel", background=p["bg"], foreground=p["fg"])
28     style.configure("TButton", background=p["bg"], foreground=p["fg"], bordercolor=p["fg"])
29     style.map("TButton", background=[("active", p["select_bg"])], foreground=[("active", p["select_fg"])])
30     style.configure("TFrame", background=p["bg"])
31     style.configure("TLabelframe", background=p["bg"], foreground=p["fg"])
32     style.configure("TLabelframe.Label", background=p["bg"], foreground=p["fg"])
33     style.configure("TNotebook", background=p["bg"])
34     style.configure("TNotebook.Tab", background=p["bg"], foreground=p["fg"])
35     style.map("TNotebook.Tab",
36             background=[("selected", p["select_bg"])],
37             foreground=[("selected", p["select_fg"])])
38     style.configure("Treeview", background=p["tree_bg"], foreground=p["fg"], fieldbackground=p["tree_bg"])
39     style.map("Treeview",
40             background=[("selected", p["select_bg"])],
41             foreground=[("selected", p["select_fg"])])
42     ThemeManager._recursive_update(root, p)
43
44     @staticmethod
45     def _recursive_update(widget, p):
46         try:
47             w_type = widget.winfo_class()
48             if w_type in ("Text", "Listbox", "Canvas"):
49                 widget.config(
50                     bg=p["text_bg"], fg=p["text_fg"],
51                     selectbackground=p["select_bg"], selectforeground=p["select_fg"],
52                 )
53             if w_type == "Canvas":
54                 widget.config(bg=p["bg"])
55             elif w_type == "Label":
56                 widget.config(bg=p["bg"], fg=p["fg"])
57         except:
58             pass
```

```
59     for child in widget.winfo_children():
60         ThemeManager._recursive_update(child, p)
```

`__init__.py`

```
1 # ui/windows package
```

`user_guide.py`

```
1 import tkinter as tk
2 from tkinter import ttk, messagebox
3
4 from core.config import GUIDE_CONTENT
5
6
7 class UserGuideWindow(tk.Toplevel):
8     def __init__(self, parent):
9         super().__init__(parent)
10        self.title("User Guide - Zip to PDF Professional")
11        self.geometry("900x600")
12        self.state("zoomed")
13
14        paned = ttk.PanedWindow(self, orient=tk.HORIZONTAL)
15        paned.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
16
17        fr_left = ttk.Frame(paned, width=250)
18        paned.add(fr_left, weight=1)
19        ttk.Label(fr_left, text="MUC LUC", font=("Segoe UI", 12, "bold")).pack(pady=(0, 10), anchor="w")
20        self.list_steps = tk.Listbox(
21            fr_left, font=("Segoe UI", 11), selectmode=tk.SINGLE,
22            activestyle="none", height=15, bd=0, bg="#f0f0f0",
23        )
24        self.list_steps.pack(fill=tk.BOTH, expand=True)
25        self.list_steps.bind("<<ListboxSelect>>", self.on_step_select)
```

```
26
27     fr_right = ttk.Frame(paned)
28     paned.add(fr_right, weight=4)
29     self.lbl_title = ttk.Label(fr_right, text="", font=("Segoe UI", 16, "bold"), foreground="#0056b3")
30     self.lbl_title.pack(pady=(0, 15), anchor="w")
31     self.txt_content = tk.Text(
32         fr_right, font=("Segoe UI", 12), wrap="word",
33         bd=0, bg="#f0f0f0", padx=10, pady=10,
34     )
35     self.txt_content.pack(fill=tk.BOTH, expand=True)
36
37     self.steps_keys = list(GUIDE_CONTENT.keys())
38     for key in self.steps_keys:
39         self.list_steps.insert(tk.END, f" {key}")
40     self.list_steps.selection_set(0)
41     self.on_step_select(None)
42
43     def on_step_select(self, event):
44         sel = self.list_steps.curselection()
45         if not sel:
46             return
47         key = self.steps_keys[sel[0]]
48         data = GUIDE_CONTENT[key]
49         self.lbl_title.config(text=data["title"])
50         self.txt_content.config(state="normal")
51         self.txt_content.delete("1.0", tk.END)
52         self.txt_content.insert("1.0", data["content"])
53         self.txt_content.config(state="disabled")
54
55
56 class WelcomeDialog(tk.Toplevel):
57     def __init__(self, parent, on_guide_callback):
```

```
58     super().__init__(parent)
59     self.on_guide = on_guide_callback
60     self.title("Welcome")
61     self.geometry("600x450")
62     self.resizable(False, False)
63     x = (self.winfo_screenwidth() - 600) // 2
64     y = (self.winfo_screenheight() - 450) // 2
65     self.geometry(f'"+{x}"+{y}')
66     self.transient(parent)
67     self.grab_set()
68
69     frame = ttk.Frame(self, padding=20)
70     frame.pack(fill=tk.BOTH, expand=True)
71     ttk.Label(frame, text="SIU IC DESIGN LAB", font=("Arial", 20, "bold"), foreground="#333").pack(pady=(10, 5))
72     ttk.Label(frame, text="Zip to PDF Professional Tool", font=("Segoe UI", 12), foreground="#666").pack(pady=(0, 20))
73
74     info_text = (
75         "Chao mung ban den voi cong cu chuyen doi ma nguon chuyen nghiep.\n\n"
76         "Cac tinh nang noi bat:\n"
77         "? Ho tro Monorepo lon, khong tran RAM.\n"
78         "? Xuat PDF chuan Audit/Review voi Metadata & TOC.\n"
79         "? Tu dong phat hien va bo qua file nhi phan.\n"
80         "? Che do xem truoc (Preview) thong minh.\n\n"
81         "Hay chon mot hanh dong de bat dau:"
82     )
83     ttk.Label(frame, text=info_text, font=("Segoe UI", 10), justify="center").pack(pady=10)
84
85     btn_frame = ttk.Frame(frame)
86     btn_frame.pack(pady=30)
87     ttk.Button(btn_frame, text="? Xem Huong Dan", command=self.open_guide, width=20).pack(side=tk.LEFT, padx=10)
88     ttk.Button(btn_frame, text="? Bat Dau Ngay", command=self.close_dialog, width=20).pack(side=tk.LEFT, padx=10)
89
```

```
90     self.var_dont_show = tk.BooleanVar(value=False)
91     ttk.Checkbutton(frame, text="Khong hien lai bang nay lan sau", variable=self.var_dont_show).pack(side=tk.BOTTOM)
92
93     def open_guide(self):
94         self.on_guide()
95         self.close_dialog()
96
97     def close_dialog(self):
98         if self.var_dont_show.get():
99             from core.config import AppConfig
100             cfg = AppConfig.load()
101             cfg["show_welcome"] = False
102             AppConfig.save(cfg)
103     self.destroy()
```